マインスイーパーで学ぶ

Bend 2 中級

標準ライブラリ・入出力・GUI・仕様と証明

プレビュー版

本書の目的は、入門編では十分に扱わなかった Bend 2 の機能を、型、引数、返り値、所有権、失敗時の挙動まで理解し、実際に使えるようになることです。マインスイーパーは学習を支える共通の題材です。本文で扱う一部の高度な機能は、GUI や外部環境が必要なため、動作条件を明示しています。

2026 年 10 月

はじめに

この本は『ライフゲームで学ぶはじめての Bend2』を読んだ人を対象とします。関数、パターンマッチ、再帰、アフィン性、do の基本は既習とします。難しいテクニックを身につけるよりも、**使える道具の一覧を把握し、必要な場面で正しい使い方を選べること**を重視します。

本書には、マインスイーパーとは直接関係のない独立したプログラムもあります。たとえばネットワーク通信や音声出力をゲームに無理に組み込むと、それらの本来の使用方法が分かりにくくなるためです。

Contents

はじめに **i**

I 標準ライブラリとデータ構造 **1**

1 Base を調べる **2**

2 数値、文字、比較を正しく選ぶ **3**

2.1 文字列の前処理と安全な変換 3

3 List と Array の詳しい使い方 **5**

3.1 List.filter と foldl を組み合わせる 5

3.2 Array.new の引数は長さではない 6

4 Map と Set を使う **7**

4.1 「マスに何かを覚えておく」には何が必要か 7

4.2 まず Set で、旗の有無を表す 7

4.3 同じ操作を Map で考える 8

4.4 重複、削除、キーの一覧 9

4.5 使う構造を選ぶ 9

5 失敗、選択肢、依存する値 **10**

5.1 0 を返せば十分だろうか 10

5.2 失敗の理由を知りたいときは Result 10

5.3 複数の値を返すことと、二種類の結果とは違う 11

5.4 Kind と数量指定は何を制限しているのか 11

II 入出力と並行処理 **12**

6 IO を読む **13**

6.1 「ゲームの規則」と「画面に出す」は別の仕事 13

6.2 どうして do の中で順番を書くのか 13

6.3 失敗し得る IO には二重の結果がある 13

6.4 何を IO にするべきか 14

7 File とリソースの受け渡し 15

7.1 File.read による読み込みとラウンドトリップ 16

8 引数、時刻、乱数、プロセス 17

8.1 環境変数は常に存在するとは限らない 17

8.2 乱数の取得と失敗の伝播 18

8.3 外部プログラムを起動する 18

9 IO.fork、Chan とタイムアウト 19

9.1 独立した IO を開始して結果を待つ 19

10 TCP、UDP と外部環境 21

III Image・App・GUI 22

11 Image と四分木 23

11.1 画面を描くには、どの単位で考えるのか 23

11.2 マインスイーパーならどう描くか 23

12 App とイベントの処理 24

12.1 画面が変わるまでを三つに分ける 24

12.2 App の view と tick 24

12.3 本物のゲームでは、状態の更新を純粋関数にする 25

13 マインスイーパーの状態 26

13.1 マスには二種類の情報がある 26

13.2 周囲の地雷数を数える 27

13.3 操作を定義する順序 28

14 表示と入力をテストする 29

14.1 クリックの位置を盤面の番号へ変換する 29

14.2 テストを分けて考える 30

IV 仕様駆動開発と形式的証明 31

15 law を先に書く 32

15.1 まず「こう動くはずだ」を書き出す 32

15.2 見かけ上もっともらしい仕様の落とし穴 33

15.3 証明済みと、動作確認済みは違う 33

16 等式、帰納法、補題と書き換え **34**
16.1 三つの状態の証明から始める 34
16.2 再帰と帰納法の対応を見る 34
16.3 等式を使い回すための操作 34
16.4 証明の信頼範囲 35

17 盤面の不変条件 **36**

18 証明の範囲と信頼境界 **37**

V 並列実行とその他の応用 **38**

19 CPU、GPU、Array.atomic **39**

20 C・JavaScript と外部連携 **40**

VI マインスイーパー統合と発展 **41**

21 一人で遊べるアプリにする **42**

22 発展：運ゲー無しマインスイーパー **43**
22.1 なぜ通常の盤面では推測が必要になるのか 43
22.2 最小の推論規則を Bend 2 で書く 43
22.3 複数の数字から推論する 45
22.4 公開情報と真の地雷配置を分離する 45
22.5 候補生成と検証を繰り返す 45
22.6 小盤面用の参考実装と検証 46
22.7 「運ゲー無し」の保証範囲 46

23 API 索引と次の学習 **47**

Part I

標準ライブラリとデータ構造

Chapter 1

Base を調べる

この章で学ぶこと：API の宣言、型、数量指定、結果を返す形を読み取る。

入門編では `import Base` を使いました。ここでは、`bend base` と `bend base Array` を使い、提供されている関数を自分で調べます。最新情報はhttps://github.com/bendlang/bend/blob/main/bend2/base.bendも参照してください。

例えば `Set.has` の戻り値は単なる `Bool` ではなく `Set() & Bool` です。「所属を調べる際に集合が消費され、再利用するには返された集合を受け取る」という設計を、この型だけから読み取れます。`Map.get` も元の辞書と値を組で返します。型名、消費する引数、戻ってくる所有物を最初に確認する習慣が重要です。

```
bend base
bend base Set
bend base Map
bend guide
```

すべての定義が通常のユーザー向け API とは限りません。`.go` や `.fin` など実装上のヘルパーも混在しています。最初は外側の公開操作を学び、実装を読む必要が生じたときに内部関数へ進みます。

演習　`Array.get`、`Set.has`、`File.open` を調べ、各関数について「入力」「結果」「失敗の有無」を記録してみましょう。

Chapter 2

数値、文字、比較を正しく選ぶ

この章で学ぶこと：整数と浮動小数点、文字列、変換時の失敗を整理する。

Nat は非負の自然数、U32 は固定幅の非負整数、F32 は近似的な浮動小数点数です。サイズの上限、演算結果、オーバーフロー、丸めが同じではありません。型が違う数をそのまま加算できるとは限らず、U32.to_nat など明示的な変換を使います。

String には split、join、trim、starts_with、to_upper などがあり、Char には文字に関する操作があります。単純な文字列の切り分けと、符号化や文字境界の意味を伴う操作は区別します。文字列を整数として読む U32.read は失敗し得るため、返り値の Maybe を確認します。

2.1 文字列の前処理と安全な変換

ユーザーが入力した文字列には、前後に空白が付いていることがあります。最初に String.trim で前後の空白を取り除き、次に U32.read で非負整数に変換する例を動かしましょう。

サンプル：空白を除去して文字列を数値に変換する

```
import Base

# String.read-like conversion can fail. Whitespace is removed first.
def parsed_number(value: Maybe<&2, U32>) -> U32:
  match value:
    case Some{n}:
      n
    case None{}:
      0

def main() -> U32:
  cleaned = String.trim(" 42 ")
  parsed_number(U32.read(cleaned))
```

U32.read は Maybe 型を返します。成功は Some、失敗は None です。このサンプルでは挙動を確認しやすいよう、失敗時に 0 を返しますが、**実際のアプリでは「失敗した 0」と「入力された 0」を混同しない設計**にしましょう。変換結果を Maybe のまま呼び出し元へ渡す方が明確です。

String.split は区切り文字を Char 一文字で受け取り、文字列のリストを返します。String.join はその逆方向の処理です。split と join の名前だけを覚えるのではなく、**区切り文字の型と、空入力のときの結果**を API と実行例で確認してください。保存ファイルの 1 行を String.split で分解する場合も、項目数と数値変換の失敗を調べる必要があります。

Cmp は大小の比較結果を表し、Bool は真偽を表します。「大小の三種類」と「条件式の二種

類」を使い分けると、ソートや検索の API を読みやすくなります。

演習　文字列`"42"`と`"x"`を数値として読む例を作り、成功・失敗の結果の型と中身を説明してみましょう。

Chapter 3

List と Array の詳しい使い方

この章で学ぶこと：構造の違いと、関数の選択が所有権に与える影響を理解する。

`List` は `Nil` と `Con` からなる再帰的な鎖です。`map`、`filter`、`foldl`、`foldr`、`sort` などが用意されています。変換したリストを作るとき、入力要素の型が `Type` か `Data` かによって、要素を何度も使えるかが変わります。

サンプル：リストを作り要素数を求める

```
import Base

# A linked list, and its length computed by the Base library.
def main() -> Nat:
  xs = List.replicate(U32, 3n, 7)
  List.length(&2, U32, xs)
```

このプログラムは `List.replicate` で値を 3 個並べたリストを作り、その長さを返します。`List.length` の引数には、数量と要素型が必要です。先頭への追加には `<>` という記法も使えます。

3.1 List.filter と foldl を組み合わせる

`List.filter` は条件を満たす要素だけを残します。`List.foldl` は先頭から値を集計し、最後の一つの値にまとめます。「抽出」と「集計」は別の操作です。

サンプル：小さい整数だけを抽出して合計する

```
import Base

# List.range returns List<&2, Nat>. Filter keeps that quantity.
def small(n: Nat) -> Bool:
  Nat.is_lt(n, 3n)

def sum(acc: Nat, n: Nat) -> Nat:
  Nat.add(acc, n)

def main() -> Nat:
  xs = List.range(4n)
  ys = List.filter(~Nat, ~small, xs)
  List.foldl(~&2, ~Nat, ~Nat, ~sum, ys, 0n)
```

`List.range(4n)` は 0 から 3 までのリストを作り、`List.filter` で 3 未満の値を残します。その合計は `3n` です。`foldl` は「現在までの合計」と「次の要素」を受け取る関数を引数にします。

List.filter は List<&2, A> を扱う一方、List.map は List<A> を受け取るなど、同じ List に見えても数量指定が異なる API があります。呼び出せないときは、引数の個数だけでなく**型の数量指定**まで比較しましょう。型の制約に合わないリストを強引に再解釈するのではなく、選ぶ関数やデータの構造を見直します。

Array はインデックスを使う構造です。get と set のほか、swap、clone、fork、join、to_list、map などがあります。「読むだけ」のつもりでも所有権付き配列を消費するので、返された配列を引き継ぎます。複製やフォークは処理・メモリのコストを伴います。

3.2 Array.new の引数は長さではない

Array.new の第二引数は「作る要素数」そのものではなく、二分木状の配列の深さです。深さを d とすると要素数は 2^d です。たとえば Array.new(U32, 3n, 0) は 8 要素を作ります。

サンプル：8 要素の配列の 2 番目と 7 番目を書き換える

```
import Base

# Array.new(d) produces an array of 2^d elements.
def read_and_sum(r: Array<U32> & U32) -> U32:
  (a, x) = r
  x

def main() -> U32:
  a = Array.new(U32, 3n, 0)
  b = Array.set(U32, a, 2, 17)
  c = Array.set(U32, b, 7, 25)
  x = read_and_sum(Array.get(U32, c, 2))
  x
```

Array.set は更新した配列を返します。以前の配列をそのまま後続で使うのではなく、返された配列を引き継ぎます。Array.get は配列と読み出した値の組を返します。例では配列を再使用しないため捨てていますが、次の読み出しや更新を行う場合は受け取ってください。インデックスが範囲外の場合の扱いも、公開 API の実装を確認しましょう。

演習　同じ数値列を List と Array の両方で表し、要素の変更、走査、複製が必要な場合の選び方を比較してみましょう。

Chapter 4

Map と Set を使う

この章で学ぶこと：どのデータを記録したいかから、Map と Set を選び、更新後の所有権まで扱う。

4.1 「マスに何かを覚えておく」には何が必要か

前巻のライフゲームでは、各マスの状態を配列に入れました。マインスイーパーでも配列は使えますが、例えば**旗を立てた場所だけを記録する**なら、すべてのマスに値を用意する必要はありません。座標`"2,3"` が旗の集合に含まれるかだけ分かれば十分です。

一方、**マスごとの数字や戦績を対応付ける**ときには、場所と値の組を覚えたいはずです。前者には `Set`、後者には `Map` が向いています。どちらも `Base` にあるため、独自の探索用データ構造を作る前に使い方を確認します。Bend 2 の両方の構造は文字列をキーとして使います。

4.2 まず Set で、旗の有無を表す

サンプル：集合へ座標を追加して調べる

```
import Base

# Set contains String keys; membership returns the set as well.
def membership(r: Set() & Bool) -> U32:
  (again, exists) = r
  match exists:
    case True{}:
      1
    case False{}:
      0

def main() -> U32:
  s0 = Set.new()
  s1 = Set.add(s0, "2,3")
  membership(Set.has(s1, "2,3"))
```

この例では`"2,3"` を追加し、`Set.has` が真を返すことを確かめています。実行結果は `1` です。読者がまず注目したいのは「どうして所属判定の結果が単なる `Bool` ではないのか」という点です。

`Set.has` の結果は `Set() & Bool` です。**調べた後も集合を引き続き使えるよう、集合自体が返ってくる**のです。コード内で返り値を `(again, exists)` に分け、今回使わない `again` を捨

てています。続けて旗を更新したいなら、その返された集合を次の操作に渡します。

次の例は、調べた結果に応じて追加と削除を選びます。

サンプル：同じ場所の旗を二回切り替える

```
import Base

# Keyed squares: Set operations are affine, so the new set is passed on.
def from_membership(r: Set() & Bool, key: String) -> Set():
  (s, present) = r
  match present:
    case True{}:
      Set.del(s, key)
    case False{}:
      Set.add(s, key)

def toggle_mark(s: Set(), +key: String) -> Set():
  from_membership(Set.has(s, key), key)

def main() -> Nat:
  s0 = Set.new()
  s1 = toggle_mark(s0, "2,3")
  s2 = toggle_mark(s1, "2,3")
  Set.size(s2)
```

一回目の操作で"2,3" に旗が付き、二回目で取り除かれるため、最後の集合の要素数は 0n になります。0n は Nat 型の 0 です。結果を実行してから追うのではなく、**一行ごとにどの集合が次の行へ渡されるか**を紙に書いてみるとアフィン性が理解しやすくなります。

4.3 同じ操作を Map で考える

Set に記録できるのはキーの存在です。Map ではキーに値を割り当てます。例えばキー"wins" に値 7 を保存できます。

サンプル：辞書に値を保存して取り出す

```
import Base

# Map.get returns a rebuilt map and its value.
def take_value(r: Map<&2, U32> & U32) -> U32:
  (again, value) = r
  value

def main() -> U32:
  m0 = Map.new(&2, U32)
  m1 = Map.set(&2, U32, m0, "wins", 7)
  take_value(Map.get(U32, 0, m1, "wins"))
```

Map.new では値の型と数量を指定します。Map.set は更新された辞書を返し、Map.get は**辞書と取り出した値の組**を返します。この例の実行結果は 7 です。

ここで気になるのは、辞書に存在しないキーを読んだときの振る舞いでしょう。Map.get には**既定値**を渡す引数があり、見つからなければその値が返ります。例えば既定値が 0 なら、戻り値の 0 が「登録済みの 0」なのか「未登録」なのか、この結果だけでは分かりません。Map.has で存在を確かめるか、値自体を Maybe 等にする設計を選びます。この違いは、保存データの欠落をエラーにするかどうかに直結します。

4.4 重複、削除、キーの一覧

サンプル：重複を除く Set と、キーを列挙する Map

```
import Base

# Set.add discards duplicates; Map.keys returns a list of String keys.
def main() -> Nat:
  s0 = Set.new()
  s1 = Set.add(s0, "0,0")
  s2 = Set.add(s1, "0,0")
  s3 = Set.add(s2, "1,1")
  s4 = Set.del(s3, "0,0")
  m0 = Map.new(&2, U32)
  m1 = Map.set(&2, U32, m0, "wins", 3)
  m2 = Map.set(&2, U32, m1, "games", 7)
  keys = Map.keys(&2, U32, m2)
  a = Set.size(s4)
  b = List.length(&2, String, keys)
  Nat.add(a, b)
```

同じキーを集合へ二度追加しても、要素数は増えません。一方、辞書の `Map.keys` は、格納されたキーのリストを返します。例の最後の結果は、集合の要素数 1 と辞書のキー数 2 を合わせた `3n` です。

ただし、`Map.keys` の返す順番が登録順だとは仮定しないでください。順番に意味がある画面を作る場合は、別のリストで順番を管理するか、明示的に並べ替えます。

4.5 使う構造を選ぶ

今回の題材では、地雷の位置と旗の位置は `Set`、名前付き設定や戦績は `Map`、盤面全体のマスの並びは `Array` という選び方が考えられます。必ずこの組合せにしなければならないわけではありません。**何を検索したいか、更新した後どの値が戻るか**を基準に選べば、構造の名前だけを覚えるより応用が利きます。

演習 `16_map_set_api.bend` で削除の行を一つ外した場合、最後の値はいくつになりますか。実行前に予想し、変更後の値と一致するか確かめましょう。

Chapter 5

失敗、選択肢、依存する値

この章で学ぶこと：「値がない」と「操作を受け付けない」を区別し、型を選べるようになる。

5.1　0 を返せば十分だろうか

座標を調べる関数が 0 を返したとします。これは「周囲の地雷数が 0」なのか、「その座標が見つからなかった」のか、関数の外からは判断できません。値そのものと、値を得られなかったという状況を別々に表す必要があります。

前巻で学んだ `Maybe` は、`Some{value}` と `None{}` を区別します。存在しなかった理由までは必要ない場合に便利です。例えば数値として読めない文字列を `U32.read` へ渡すと `None{}` になり得ます。既出の `15_string_parse.bend` をもう一度見て、成功と失敗でどの分岐を通るか確認しましょう。

5.2　失敗の理由を知りたいときは Result

マインスイーパーでは「すでに開いたマスには旗を付けられない」という操作を考えます。失敗したことだけでなく、なぜ操作できないかを伝えたいとき、`Result` を使います。

サンプル：旗を付ける操作の成功と失敗

```
import Base

# Reject flagging an already revealed square.
# This is a pure game-rule failure, not an IO error.
type Cell is Data:
  Hidden{}
  Flag{}
  Open{}

def flag(s: Cell) -> Result<&2, &2, String, Cell>:
  match s:
    case Hidden{}:
      Done{Flag{}}
    case Flag{}:
      Done{Flag{}}
    case Open{}:
      Fail{"an open square cannot be flagged"}

def success(r: Result<&2, &2, String, Cell>) -> U32:
  match r:
```

```
    case Done{cell}:
      1
    case Fail{reason}:
      0

law hidden_can_be_flagged:
  {flag(Hidden{}) == Done{Flag{}} : Result<&2, &2, String, Cell>}

def hidden_can_be_flagged():
  {==}

def main() -> U32:
  a = success(flag(Hidden{}))
  b = success(flag(Open{}))
  (a + b : U32)
```

`flag` は、閉じたマスなら `Done{Flag{}}` を、開いたマスなら `Fail{...}` を返します。これは**純粋関数で判定したゲームの規則**です。ファイルへの書き込み失敗のような IO エラーとは分けて考えます。コードの `success` は成功なら 1、失敗なら 0 に変えているため、`main` は `1` を返します。

`Result` を読む際は、**成功の値の型**と**失敗の理由の型**を別々に確かめましょう。この例では成功の値は `Cell` で、失敗の理由は `String` です。`Result` の宣言には数量引数も含まれているため、型を記すときは `Result<&2, &2, String, Cell>` のようになります。こうした追加の型引数を適当に省略すると、型検査で失敗することがあります。

5.3 複数の値を返すことと、二種類の結果とは違う

`A & B` は二つの値をまとめた組です。`Map.get` が「辞書と読み出した値」を返すのは、**両方を同時に持っている**からです。一方、`Either` の `Inl` と `Inr` は、**どちらか一方が入っている**ことを表します。`Result` はこの発想を成功・失敗という用途に特化した型です。

`Sigma` はさらに一般的な組で、**最初の値によって次の値の型が決まる**場合に使います。普通の二つの数を返すだけなら依存型にする必要はありません。この段階では「普通の組と、依存する組を区別できる」ことを目標にし、実際の存在命題と依存型は証明の Part で扱います。

5.4 Kind と数量指定は何を制限しているのか

ここまでの `Map.get` や `List.filter` では、`&1`・`&2`・`Type`・`Data` などを見ました。これらは装飾ではありません。ある値を複数回使ってよいか、使った後に戻す必要があるかに関係します。`Kind` はそうした型の分類を表し、`Quant` は数量を指定する仕組みです。

最初に覚えるべき手順は三つです。関数の引数の型を読むこと、返り値で所有権が戻るかを見ること、複製が必要なら適切な数量指定を使うことです。すべての種類の数量を暗記するより、`Set.has` と `File.read` のような実例で意味を確認すると混乱しません。

演習　「座標を探したが見つからない」「保存が失敗したので理由を知らせたい」「辞書と読み出した値を返したい」それぞれに、`Maybe`、`Result`、組のどれが適切かを選び、理由を説明してみましょう。

Part II

入出力と並行処理

Chapter 6

IO を読む

この章で学ぶこと：値を計算する処理と、画面・ファイルなど外界に作用する処理を分ける。

6.1 「ゲームの規則」と「画面に出す」は別の仕事

前章の `flag` は、引数が同じならいつ呼んでも同じ結果を返します。画面がなくても検査できる**純粋関数**です。ところが、画面に文章を表示する、ファイルを書き換える、乱数を取得するといった操作は、プログラムの外に影響したり、外の状態に左右されたりします。このような**副作用を伴う計算**を表すために、`IO<A>` を使います。

`IO.print("hello")` は `IO(Unit)` です。文章を表示する仕事を表し、特に返す値がないことを `Unit` で表現しています。対して `IO.args()` は `IO(List<String>)` です。起動時の引数をリストとして受け取ります。**IO の括弧の中身は、作用の結果として得る値の型**です。

6.2 どうして do の中で順番を書くのか

マインスイーパーの設定を保存するなら、まずファイルを開き、次に書き込み、最後に閉じます。順番を入れ替えると意味が変わる処理です。`do IO<Unit>:` は、こうした順序を読みやすく書くためにあります。

すでに使った `08_args.bend` を見直すと、`<-`の左側で得た値を、次の行へ渡しています。これが「外部の処理が返した結果を受け取り、次の処理を続ける」という流れです。`IO.pure` は、外へ作用せず値を `IO` の結果として返すための操作で、後の `IO.fork` の実例にも登場します。

6.3 失敗し得る IO には二重の結果がある

`File.open` や `IO.random_u32` は、成功するとは限りません。そのため `IO(Result<...>)` のように、**外部の処理**と**その成功・失敗**を両方型に書きます。`IO.try` は、結果が `Done` なら値を取り出し、`Fail` なら後続の計算へ進まず失敗を伝えます。

ここで重要なのは、`IO.try` を使えば必ず回復できるわけではないということです。失敗時に別の保存先を試すなど、回復する方針が必要なら、`Result` の値を直接受け取って `match` で分岐します。第 7 章で実際のファイル操作を見ます。

6.4 何を IO にするべきか

旗を付けられるかの判定、周囲の地雷数、勝敗の規則は純粋関数にできます。一方、結果を画面に表示する、設定を保存する、実際の時刻を読む、といった境界の仕事は IO です。両者を分けると、ゲームの規則を `law` で述べたり、自動テストで調べたりできます。

演習 「マスを開いた後に地雷数を調べる」「戦績をファイルへ保存する」「マウスクリックを待つ」の三つを、純粋な部分と IO が必要な部分に分けてみましょう。

Chapter 7

File とリソースの受け渡し

この章で学ぶこと：ファイルの開閉、文字・バイト列、失敗後の扱いを学ぶ。

`File.open(path,mode)` は `IO(Result<...,File>)` を返します。開けないときはハンドルがありません。`File.read(file,max)` は、**継続して使う File と読み込み結果の組**を返します。書き込み操作も同様に結果を確認します。最後に `File.close` を呼びます。

サンプル：ファイルへ文字列を書き込む例

```
import Base

# File operations return the handle so it can be used or closed exactly once.
# Run in a writable directory. This writes "sample_board.txt".

def close_after_write(r: File & Result<&1, &1, U32 & String, Unit>) -> IO(Unit):
  (file, status) = r
  match status:
    case Done{value}:
      File.close(file)
    case Fail{error}:
      do IO<Unit>:
        Unit <- File.close(file)
        IO.die(Unit, 1, "Could not save sample_board.txt")

def main() -> IO(Unit):
  do IO<Unit>:
    file: File <- IO.try(File, File.open("sample_board.txt", "w"))
    written: File & Result<&1, &1, U32 & String, Unit> <- File.write(file,
     "board-size=4\nmines=1,1\n")
    close_after_write(written)
```

この例は作業ディレクトリに `sample_board.txt` を作ります。ファイルを開けないときには `IO.try` が処理を失敗として終了します。`File.write` は元のファイルハンドルを消費し、書き込み結果と**継続して利用できるハンドル**を返します。`close_after_write` は `Done`・`Fail` を分け、どちらもハンドルを閉じます。

ファイルへのアクセスは、成功した書き込みだけを想定してはいけません。開くときの権限不足、保存先の不足、書き込みの失敗などを検査します。保存後にファイルを読み戻すと、期待した内容が書き込まれたか確かめられます。

7.1 File.read による読み込みとラウンドトリップ

保存の次は読み込みです。`File.open(path, "r")` で読み取り用に開き、`File.read(file, 4096)` で最大 4096 バイトを受け取ります。読み込みの結果は `Result` に包まれており、成功時の `String` とファイルハンドルを取り出してから、必ず `File.close` します。

サンプル：保存した文字列を読み取る例

```
import Base

# Reads the file written by 12_file_write.bend.
# Every branch closes the affine File handle.

def close_after_read(r: File & Result<&1, &1, U32 & String, String>) -> IO(String):
  (file, status) = r
  match status:
    case Done{contents}:
      do IO<String>:
        Unit <- File.close(file)
        return contents
    case Fail{error}:
      do IO<String>:
        Unit <- File.close(file)
        IO.die(String, 1, "Could not read sample_board.txt")

def main() -> IO(Unit):
  do IO<Unit>:
    file: File <- IO.try(File, File.open("sample_board.txt", "r"))
    result: File & Result<&1, &1, U32 & String, String> <- File.read(file, 4096)
    contents: String <- close_after_read(result)
    IO.print(contents)
```

この例は `12_file_write.bend` の実行後に動かします。ファイルが存在しなければ読み込みは失敗として終了します。保存した内容を読み戻して元の文字列と比較すると、保存・復元を一組としてテストできます。これを**ラウンドトリップテスト**と呼びます。

バイト列を直接扱う場合には、次の別の操作を使います。

- `File.read_bytes`：バイト列を読み込みます。
- `File.write_bytes`：バイト列を書き込みます。

`File.read` が一度の呼び出しで必ずファイル全体を返すとは限りません。大きいファイルでは残りを繰り返し読む設計が必要です。文字列とバイト列、ファイルの終端、指定した読み込み最大長の関係も API の型から確認しましょう。

保存形式には、まず単純な行単位のテキストを使うとよいでしょう。地雷位置、開いたマス、旗位置、盤面サイズ、版番号を別々に記録し、読み込み時に入力の妥当性をチェックします。任意の外部ファイルを信頼してはいけません。

演習 「存在しないファイル」「空ファイル」「項目が欠けたファイル」の三種類を用意し、それぞれどう扱うべきか設計してみましょう。

Chapter 8

引数、時刻、乱数、プロセス

この章で学ぶこと：外部情報を受け取る方法と、再現できるテストの作り方を学ぶ。

`IO.args` は起動時の引数をリストとして受け取ります。設定ファイルの位置や盤面のサイズを指定する例で使えます。`IO.get_env` は環境変数を読みます。

サンプル：引数の個数を表示する

```
import Base

# IO.args returns a list including the executable or source path.
def main() -> IO(Unit):
  do IO<Unit>:
    xs: List<String> <- IO.args()
    IO.print(Nat.show(List.length(&1, String, xs)))
```

8.1 環境変数は常に存在するとは限らない

環境変数は OS や起動方法によって変化します。`IO.get_env("PATH")` はパス検索の設定を読む例ですが、必ず値があると仮定してはいけません。

サンプル：環境変数を読み取り長さを表示する

```
import Base

# The environment may not contain the requested name.
def env_length(value: Result<&1, &1, U32 & String, String>) -> Nat:
  match value:
    case Done{s}:
      String.length(s)
    case Fail{problem}:
      0n

def main() -> IO(Unit):
  do IO<Unit>:
    result: Result<&1, &1, U32 & String, String> <- IO.get_env("PATH")
    IO.print(Nat.show(env_length(result)))
```

`IO.get_env` の結果は `Result` です。`Done` の場合は文字列を受け取り、`Fail` の場合は例のように既定動作へ分岐できます。エラー内容が必要なら、`Fail` の値を捨てずに利用します。環境変数の中身をそのままログに出すと、パスワードなどの機密情報を漏らす可能性があります。この例では値そのものではなく長さだけを出力します。

`IO.now` は現在時刻、`IO.sleep` は時間待ち、`IO.random_u32` は乱数を扱います。乱数を使ったテストは、実際の乱数に依存させず、固定した地雷配置を入力にする方法と分けて実施します。

8.2 乱数の取得と失敗の伝播

`IO.random_u32()` の型は `IO(Result<..., U32>)` であり、単に `U32` が返るわけではありません。外部機能が失敗する可能性を明示するためです。`IO.try` を使うと成功した `U32` を取り出せますが、失敗したときは後続の `IO.print` へは進みません。

サンプル：乱数を取得して表示する

```
import Base

# The random-number effect may fail, hence the Result inside IO.
def main() -> IO(Unit):
  do IO<Unit>:
    random_value: U32 <- IO.try(U32, IO.random_u32())
    IO.print(U32.show(random_value))
```

この例は毎回値が変化するため、数値そのものを固定して比較するテストには不向きです。ソルバーの正しさは固定した盤面でテストし、ランダム盤面の生成はその後に確認しましょう。

8.3 外部プログラムを起動する

`Process.run` は外部プログラムを起動します。引数のリスト、標準入力へ渡す文字列、受け取り可能な出力の最大量、タイムアウトを指定します。

サンプル：外部プログラムを起動して終了コードを確認する

```
import Base

# Launch a harmless OS program and examine its exit code.
def exit_status(value: U32 & (String & String)) -> U32:
  (code, outputs) = value
  code

def main() -> IO(Unit):
  do IO<Unit>:
    result: U32 & (String & String) <- IO.try(U32 & (String & String),
    Process.run("/usr/bin/true", Nil{}, "", 1024, 3000))
    IO.print(U32.show(exit_status(result)))
```

この例は Unix 系環境の `/usr/bin/true` を実行します。このコマンドは何も表示せずに成功の終了コード 0 を返します。Windows などでは同じパスは使えないため、実行環境に応じてプログラムを選んでください。

`Process.run` は `IO(Result<...>)` 型の結果を返します。プロセスの**起動そのものが失敗した場合**と、**起動したプログラムが 0 以外の終了コードで終わった場合**は違います。前者は `Fail` で、後者は成功結果内の終了コードで区別します。返り値には標準出力と標準エラー出力も含まれます。ここでは終了コードだけを取り出しますが、利用者に必要な情報を捨ててはいけません。

演習　引数が不足するときは既定の盤面を使うか、理由を表示して終了するか、用途に応じて選んでみましょう。

Chapter 9

IO.fork、Chan とタイムアウト

この章で学ぶこと：独立した仕事を非同期に実行し、結果の受信と終了処理を理解する。

CPU の二つの関数呼び出しを並列に実行する記法と、IO 処理を並行させる操作は別です。`IO.fork` は IO を開始して結果を受け取るチャネルを返し、`IO.join` が結果を待ちます。`Chan.new`、`Chan.send`、`Chan.recv` はメッセージの受け渡しに使います。

`Chan.recv` は閉じられたチャネルから値が得られない場合、`None` を返すことがあります。`IO.within` など時間制限を扱う API は、時間内に完了しなかった場合の結果を確認して使います。待ち手だけを途中で止めたとき、仕事がどうなるかも仕様と実装で確認する必要があります。

9.1 独立した IO を開始して結果を待つ

`IO.fork` は IO 計算を始め、その結果を受け取るための `Chan` を返します。`IO.join` はその結果を待ち、チャネルを閉じます。最も小さな例では、別の IO 計算が返す値を受け取るだけでも利用できます。

サンプル：IO.fork で開始し IO.join で結果を受け取る

```
import Base

# fork returns a channel for the result; join waits and closes it.
def main() -> IO(Unit):
  do IO<Unit>:
    chan: Chan(U32) <- IO.fork(U32, IO.pure(U32, 7))
    value: U32 <- IO.join(U32, chan)
    IO.print(U32.show(value))
```

この例の結果は 7 です。これだけでは、CPU が別スレッドで実際に動いたことや処理が高速化したことを意味しません。`IO.pure` は、外部作用を起こさずに値を IO 計算の結果とする関数です。

`Chan.send` の結果は、送信できた場合の `Done` か、チャネルが閉じた場合の `Fail` です。`Chan.recv` は値が届いた場合の `Some` か、閉じた後で受信できない場合の `None` を返します。`IO.join` が閉じられたチャネルから値を受け取れない場合は失敗終了します。適切に結果を待ち、チャネルを閉じる責任を曖昧にしないようにしましょう。

`IO.within` は、指定時間内に結果が得られたかを `Maybe` として返します。注意すべき点として、タイムアウトしたからといって**遅れた仕事が自動的に取り消されるわけではありません**。処理が止まらない可能性がある用途では、個別の終了・中断方法も検討します。

この章では操作の型と意味を独立例で学び、マインスイーパーでは自動保存やタイマーの用

途と結び付けます。最初から複雑なワーカープールを作る必要はありません。

演習　送る側が終了したとき、受信する側はいつ処理を終えるべきか、短い手順を書いてみましょう。

Chapter 10

TCP、UDP と外部環境

この章で学ぶこと：接続の確立、失敗、リソース開放を理解する。

`TCP.listen`、`TCP.accept`、`TCP.connect`、`TCP.send`、`TCP.recv` は接続型の通信です。`UDP.bind`、`UDP.send_to`、`UDP.recv_from` はデータグラムをやり取りします。通信には相手が応答しない場合や、受信データが途中で切れる場合があります。

どちらも `Result` を調べ、`Socket` や `Listener` の所有権を返り値で受け渡します。ローカルホストの最小例から始め、外部ネットワークへの接続は別途許可を得た環境で行います。`Process.run` や音声・ファイル IO と同様、ホスト OS で使用できる機能に影響されます。

通信はマインスイーパーへ必須ではありません。本書では「何ができ、どの型を返し、どこで失敗するか」を独立した学習事項とします。

演習　TCP と UDP で「送信成功」の意味が異なるのはなぜか、通信が届かない場合を考えて説明してみましょう。

Part III

Image・App・GUI

Chapter 11

Image と四分木

この章で学ぶこと：画面を「図形部品」ではなく再帰的な画像として捉え、Pix と Qua を使い分ける。

11.1　画面を描くには、どの単位で考えるのか

普通の GUI ではボタンやテキスト欄などの部品を並べる場面が多いでしょう。しかし Bend 2 の `App` では、まず**現在の状態から画像を作る**方法を理解します。そこで登場するのが `Image` です。

画面全体を一色にしたいなら、すべての画素を一つずつ並べる必要はありません。一色の正方形を `Pix` で表します。一方、左上と右上で色を変えたいなら、四つの領域に分ければよさそうです。これを繰り返し使える構造が `Qua` です。

```
Pix{0x002244}
Qua{Pix{0}, Pix{0xffffff}, Pix{0xffffff}, Pix{0}}
```

最初の例は全体が一色です。次の例は左上と右下が黒、右上と左下が白の四つの領域に分かれています。`Qua` の引数の順番は**左上、右上、左下、右下**です。上下や左右を取り違えると、画像が意図しない配置になります。

11.2　マインスイーパーならどう描くか

盤面は正方形のマスが並ぶ構造なので、マスを画像へ変換し、それらを組み合わせるのが自然です。閉じたマスと開いたマスの色、地雷や旗の模様などを考えます。最初は色だけで状態を区別し、動作を理解してから描画を細かくすると、本来の目的から外れずに進められます。

画像を作ることと、ウィンドウを開くことは別です。`Pix` や `Qua` は画像を表す値を組み立てます。ウィンドウを開いて画面に表示するのは、次章の `App.run` の役割です。

演習　四つのマスのうち左上だけを赤、他を黒にする `Qua` の構成を考えましょう。どの位置にどの `Pix` を置くか、先に紙へ描いてからコードにしてみましょう。

Chapter 12

App とイベントの処理

この章で学ぶこと：「今の状態を表示する」と「入力から次の状態を決める」を分離する。

12.1　画面が変わるまでを三つに分ける

マインスイーパーのマスをクリックしたとします。この一回の操作を、次の流れで捉えます。

1. マウスのイベントを受け取る。
2. どのマスが選ばれたかを調べ、ゲームの状態を更新する。
3. 更新後の状態から新しい画像を作る。

「クリックされたら直ちに画面の一部を塗り替える」と考えるよりも、この三つを分けた方が、イベントの処理とゲームの規則を別々に確認できます。

12.2　App の view と tick

App は view と tick という二つの処理を組み合わせます。view は状態から Image を作り、状態と画像の組を返します。tick は List<Event> を受け取り、IO(Maybe<State>) として次の状態を返します。Some で続行し、None で終了します。

サンプル：まず色の変わる最小 GUI を動かす

```
import Base

# Adapted from the official GUIDE. Requires a graphical desktop to run.
def tick(events: List<Event>, color: U32) -> IO(Maybe<U32>):
  match events:
    case Nil{}:
      IO.pure(Maybe<U32>, Some{color})
    case Con{Close{}, rest}:
      IO.pure(Maybe<U32>, None{})
    case Con{e, rest}:
      tick(rest, (color + 1 : U32))

def main() -> IO(Unit):
  App.run(~U32, ~App{+s => (s, Pix{s}), tick}, "GUI example", 256, 256, 0)
```

この例では状態は一つの `U32` で、`view` は `Pix{s}` を描きます。イベントがないときは同じ色、閉じるイベント `Close` なら終了、それ以外のイベントなら色の数値を増やします。最初からマインスイーパーの盤面を入れないのは、**App の役割だけを先に確かめるため**です。

GUI の動作にはデスクトップ環境が必要です。コンパイラの型検査に成功したことと、実際にウィンドウが表示されマウス操作できることを区別してください。

12.3 本物のゲームでは、状態の更新を純粋関数にする

次に色を増やす代わりに、ゲームの操作 `reveal` や `toggle_flag` を呼び出す設計にします。どのマスがクリックされたかは `Event` から得た位置を変換して決めます。ゲーム側の関数へは座標や操作の種類だけを渡せば、GUI を起動しなくても規則をテストできます。

`App.run` は繰り返し状態と表示を扱います。`tick` で受け取るイベントはリストなので、イベントが複数ある場合の処理順序も考える必要があります。最初は一つのイベントを正しく扱い、次に複数のイベントへ進みましょう。

演習 `07_gui_hello.bend` で `Close` を受けたときの返り値と、イベントを受けなかったときの返り値を比較してみましょう。なぜ前者が `None` なのでしょうか。

Chapter 13

マインスイーパーの状態

この章で学ぶこと：「地雷の正解」「画面に見えていること」「操作の結果」を混ぜずに設計する。

13.1　マスには二種類の情報がある

プレイヤーがまだ開いていないマスには、地雷があるかもしれません。しかし、**画面には地雷の有無を表示しません**。同じマスについて、「内部にだけある正解」と「プレイヤーが見られる情報」の二種類が存在します。これらを同じ変数として扱うと、ソルバーに正解を渡してしまうような誤りが起こります。

まず、画面に見えるマスの状態だけを作ります。

サンプル：隠れたマス、旗、公開済みマス

```
import Base

# The local rules are pure. A later chapter attaches them to a full board.
type Square is Data:
  Hidden{}
  Flag{}
  Revealed{nearby: U32}

def flip_flag(s: Square) -> Square:
  match s:
    case Hidden{}:
      Flag{}
    case Flag{}:
      Hidden{}
    case Revealed{count}:
      Revealed{count}

def is_flag(s: Square) -> U32:
  match s:
    case Flag{}:
      1
    case Hidden{}:
      0
    case Revealed{count}:
      0

def main() -> U32:
  is_flag(flip_flag(Hidden{}))
```

この例では `Hidden`・`Flag`・`Revealed` を区別します。さらに `Revealed` には周囲の地雷数を

持たせています。これは画面に見えている状態です。真の地雷位置は別の `Set` 等で持つ設計にします。

13.2 周囲の地雷数を数える

開いたマスに表示する数字は、周囲 8 か所のうち地雷が置かれている数です。では、左上の隅のマスにも 8 か所の隣接位置があるでしょうか。盤面の外側はマスではないので、実際に調べるのは 3 か所です。この境界条件を先に確認します。

サンプル：端の外へ回り込まない 8 近傍計算

```
import Base

# A 4x4 Minesweeper board; mine positions are String keys "x,y".
# Out-of-bounds neighbor coordinates must NOT wrap around the board.

def member_bit(r: Set() & Bool) -> U32:
  (s, found) = r
  match found:
    case True{}:
      1
    case False{}:
      0

def mine_at(mines: Set(), x: U32, y: U32) -> U32:
  member_bit(Set.has(mines, U32.show(x) ++ "," ++ U32.show(y)))

def guarded(+mines: Set(), x: U32, y: U32, inside_x: Bool, inside_y: Bool) -> U32:
  match inside_x inside_y:
    case True{} True{}:
      mine_at(mines, x, y)
    case _ _:
      0

def adjacent(+mines: Set(), +x: U32, +y: U32) -> U32:
  +left = (x - 1 : U32)
  +right = (x + 1 : U32)
  +up = (y - 1 : U32)
  +down = (y + 1 : U32)
  a = guarded(mines, left, up, U32.is_lt(left, 4), U32.is_lt(up, 4))
  b = guarded(mines, x, up, U32.is_lt(x, 4), U32.is_lt(up, 4))
  c = guarded(mines, right, up, U32.is_lt(right, 4), U32.is_lt(up, 4))
  d = guarded(mines, left, y, U32.is_lt(left, 4), U32.is_lt(y, 4))
  e = guarded(mines, right, y, U32.is_lt(right, 4), U32.is_lt(y, 4))
  f = guarded(mines, left, down, U32.is_lt(left, 4), U32.is_lt(down, 4))
  g = guarded(mines, x, down, U32.is_lt(x, 4), U32.is_lt(down, 4))
  h = guarded(mines, right, down, U32.is_lt(right, 4), U32.is_lt(down, 4))
  (a + b + c + d + e + f + g + h : U32)

def main() -> U32:
  +mines = Set.add(Set.new(), "1,1")
  a = adjacent(mines, 0, 0)
  b = adjacent(mines, 2, 2)
  (a + b : U32)
```

コードでは幅と高さを 4 とし、地雷位置を`"x,y"` の文字列として `Set` に入れます。`guarded` は位置が盤面内かどうかを調べてから、地雷の有無を数値にします。`Set.has` が集合と `Bool` を返す理由は第 4 章のとおりです。例では (1,1) に一つだけ地雷があり、(0,0) と (2,2) はいずれも周囲に一つ地雷があるため、結果は 2 です。

前巻のライフゲームは盤面の端が反対側へつながる**トーラス境界**でした。しかしここでは端をまたぎません。**同じ「周囲 8 マス」でもゲームの仕様で境界条件が違う**ことを意識しましょう。

13.3 操作を定義する順序

次に定義すべき操作は、旗の切替、マスの公開、地雷を踏んだときの終了、0 のマスの周囲の連鎖公開、勝利判定です。ただし、すべてを一度に実装しません。まず旗の切替のように局所的な操作を `law` で定めて検査します。それから盤面全体へ広げます。

最初のクリックで地雷を踏まない規則を採用するなら、**初手前にはまだ地雷を置かず、初手が決まってからその場所を避けて配置する**という設計も可能です。別の実装方法もありますが、どの方式であっても初手安全の条件を仕様に含める必要があります。

演習　地雷が一つだけ (0,0) にある 4×4 盤面を考えます。(0,1)、(1,1)、(3,3) に表示される数字を予想し、サンプルの座標を変更して検査しましょう。

Chapter 14

表示と入力をテストする

この章で学ぶこと：マウス座標と盤面座標を対応させ、GUI なしで確かめられる処理を増やす。

14.1　クリックの位置を盤面の番号へ変換する

マウスから得るのは画面上の座標です。しかしゲームの盤面は「上から何行目、左から何列目のマスか」で扱います。ここに変換が必要です。

例えば、32 画素のマスを横に 8 個並べた場合、画面の $x = 64$ は左から 3 列目（0 から数えると 2 列目）です。$y = 32$ なら上から 2 行目（0 から数えると 1 行目）です。盤面を 1 次元の配列で表すならインデックスは

$$\left\lfloor \frac{y}{32} \right\rfloor \cdot 8 + \left\lfloor \frac{x}{32} \right\rfloor$$

となり、この位置は 10 です。

サンプル：マウスの画素座標を盤面インデックスへ変換する

```
import Base

# Translate a mouse position to the index of an 8x8 grid of 32-pixel squares.
# Invalid positions return None rather than selecting an unrelated square.

def cell_if_valid(x: U32, y: U32, valid: Bool) -> Maybe<&2, U32>:
  match valid:
    case False{}:
      None{}
    case True{}:
      Some{U32.add(U32.mul(U32.div(y, 32), 8), U32.div(x, 32))}

def pixel_to_cell(+x: U32, +y: U32) -> Maybe<&2, U32>:
  cell_if_valid(x, y, Bool.and(U32.is_lt(x, 256), U32.is_lt(y, 256)))

def cell_or_zero(result: Maybe<&2, U32>) -> U32:
  match result:
    case Some{cell}:
      cell
    case None{}:
      0

def main() -> U32:
  a = cell_or_zero(pixel_to_cell(64, 32))
  b = cell_or_zero(pixel_to_cell(256, 32))
  (a + b : U32)
```

この例は画面の大きさを 256×256 画素としています。画面の外側 $x = 256$ を、そのまま配列の一つのマスだと思ってはいけません。`pixel_to_cell` は有効な座標なら `Some{index}`、範囲外なら `None{}` を返します。GUI から受け取った値を使う前に**範囲を確かめる**ことが重要です。

14.2 テストを分けて考える

座標変換、周囲の地雷数、旗を付けられるかの判定は、ウィンドウを開かずに実行できます。境界上、範囲外、最初と最後のマスのような例で確認しましょう。

一方、ウィンドウが実際に開くか、クリックイベントが届くか、描画した色や大きさが意図どおりかは、実際の画面が必要です。純粋関数のテストが成功しただけで、GUI の実機動作まで確認できたとは言えません。`Window` の直接操作や `Audio` は独立した API として扱い、基本の `App` を理解してから学びます。

演習　座標 (255,255)、(256,0)、(0,256) を入力した結果を予想しましょう。直前のサンプルの入力値を変え、予想と一致するか確かめてください。

Part IV

仕様駆動開発と形式的証明

Chapter 15

law を先に書く

この章で学ぶこと：仕様、実装、証明を別々の役割として考え、期待する性質を不足なく述べる。

15.1　まず「こう動くはずだ」を書き出す

旗をクリックで付け外しする関数を作る前に、どう動くべきか考えます。閉じたマスに旗を付けると旗付きになり、旗付きのマスでは旗が外れ、すでに開いたマスの表示は変わりません。これはコードを書く前に決める**仕様**です。

もう一つ、同じマスの旗を二回切り替えると元に戻るはずです。後の確認に使うため、この性質を `law` として書いておきます。

サンプル：旗操作の定義、法則、証明

```
import Base

type Cover is Data:
  Hidden{}
  Marked{}
  Open{}

def toggle(c: Cover) -> Cover:
  match c:
    case Hidden{}:
      Marked{}
    case Marked{}:
      Hidden{}
    case Open{}:
      Open{}

law toggle_twice:
  for c: Cover
  {toggle(toggle(c)) == c : Cover}

def toggle_twice(c):
  match c:
    case Hidden{}:
      {==}
    case Marked{}:
      {==}
    case Open{}:
      {==}
```

```
law opened_stays_open:
  {toggle(Open{}) == Open{} : Cover}

def opened_stays_open():
  {==}
```

この例の `toggle` は三つの状態で場合分けします。次の `law toggle_twice` は、どの `Cover` でも二回の操作の後は元に戻るという**命題**です。さらに同名の `def toggle_twice` で、三つの状態に分けた**証明**を記述します。`for c: Cover` は、任意の `Cover` 値を対象にする指定です。

15.2 見かけ上もっともらしい仕様の落とし穴

ここで「二回で戻る法則を証明できたから、旗操作は完成だろうか」と考えてみます。実は、**何も変更しない関数**でも二回で元に戻ってしまいます。つまり、この法則は正しいけれど、それだけでは弱すぎます。

そこで「閉じたマスに操作すると旗付きになる」「旗付きでは閉じた状態へ戻る」「開いたマスは変更しない」も別々に主張します。具体的な場合についての小さな `law` をまず検査し、その後、任意の状態に関する法則へ進むと理解しやすくなります。

`22_result_and_game.bend` では、開いたマスに旗を付けようとすると `Fail` になるようにしています。これは `toggle` とは**異なる操作の仕様**です。二つの関数を混ぜず、どちらの振る舞いを採用するか設計段階で決めることが大切です。

15.3 証明済みと、動作確認済みは違う

```
bend examples/04_flag_laws.bend --verdict
```

これは学習用 ZIP を展開したフォルダーで実行するコマンドです。`--verdict` で法則の検査結果を確認します。返り値の例だけを実行したときの成功と、すべての値に対する法則が証明されたことは同じではありません。また、この章の法則を証明しても、ゲーム全体の勝敗判定や GUI の入力まで正しいとは限りません。

演習　二回の旗切替で元に戻るという法則と、閉じたマスが旗付きになるという法則の違いを、何もしない関数を例に説明してみましょう。

Chapter 16

等式、帰納法、補題と書き換え

この章で学ぶこと：小さな場合分けから帰納的なデータの証明へ進み、どの部分を証明したか把握する。

16.1　三つの状態の証明から始める

前章の `Cover` は取り得る状態が三種類です。`match` でそれぞれ調べれば、有限個の場合分けで証明できます。`toggle_twice` を読んで、各 `case` でどの式が左右同じになるか考えてみましょう。`{==}` は、その場で確認したい等式を証明するための記法です。

しかし、`List` のように長さがいくつでもあり得る値では、すべての要素数を個別に列挙できません。そこで**数学的帰納法**を使います。

16.2　再帰と帰納法の対応を見る

`List` が空なら性質を確かめられるとします。さらに、残りの部分 `t` について性質が成り立つと仮定したとき、先頭に一要素 `h` を付けた `h <> t` についても成り立つことを示せれば、任意の長さで証明できます。

この考え方は、前巻の `Nat` に関する帰納法の例にも現れます。`Nat` の `On` が基底部、`1n+p` が再帰部です。再帰的な定義と、帰納的な証明の形が似ていることを確認してください。証明の呼び出しが停止しないと、根拠のない命題まで受け入れられてしまうため、終了性の条件は重要です。

16.3　等式を使い回すための操作

証明には一つの計算だけでは足りないことがあります。`Equal.sym` はすでに得た等式の向きを逆にし、`Equal.trans` は二つの等式をつないで新しい等式を導くのに使います。例えば $a = b$ と $b = c$ があれば、$a = c$ と分かります。

複雑な等式を一度に解こうとせず、「まずこの補題を示す」「その補題を別の場所で使う」という分割が読みやすい証明につながります。式変形にどの補題が必要か、紙上で中間式を書いてから証明コードに移すとよいでしょう。

16.4 証明の信頼範囲

`Sigma` や `Exists` などの存在命題では、値の存在と、その値が満たす性質を一緒に表します。ただし、型が通ること、プログラムをいくつか実行して期待値を得ること、`law` の証明が検査されることは、異なる水準の確認です。`@unsafe` や外部効果を含む実装についても、何が保証されたかを個別に検討しなければなりません。

```
bend examples/04_flag_laws.bend --verdict
bend examples/22_result_and_game.bend --verdict
```

`?TODO` などの未完成部分を残したまま、証明が終わったと扱わないでください。

演習 $a = b$ と $b = c$ の等式があるとき、$a = c$ を導く前に、双方の式に現れる中間値を見つけてみましょう。

Chapter 17

盤面の不変条件

この章で学ぶこと：局所的な操作の法則から盤面全体の仕様へ進む。

盤面全体の仕様を考えるとき、地雷位置の集合は操作によって変化しない、公開済み集合は通常の公開操作では減らない、表示する数字は周囲 8 マスの地雷数に等しい、などを区別します。

$$\mathrm{mines}(\mathrm{reveal}(b,p)) = \mathrm{mines}(b), \qquad \mathrm{open}(b) \subseteq \mathrm{open}(\mathrm{reveal}(b,p)).$$

これらは**目指す仕様**であり、この初稿で盤面全体を証明済みという意味ではありません。実装後、座標変換、隣接位置の列挙、公開操作、勝敗判定のそれぞれの補題を作る必要があります。

仕様を `LAWS.bend`、実装を別ファイル、証明を `PROOF.bend` に分けると、どこが要求でどこが検証かを整理できます。最初は同じファイルで小さな法則を試し、慣れてから分離します。

演習　一手目が安全であることを命題として書くとき、どんな入力と結果が必要か説明してみましょう。

Chapter 18

証明の範囲と信頼境界

この章で学ぶこと：保証できた性質と保証できなかった性質を正確に報告する。

地雷数の局所規則を証明しても、GUI が正しいマスを選ぶ保証や、保存データが必ず復元できる保証にはなりません。ファイル・ネットワーク・乱数・GPU・FFI など外部環境を含む部分は、純粋計算の形式的証明と別の検証が必要です。

依存型は入力に関する性質を型で表現する助けになりますが、「型注釈があるから必ず正しい」わけではありません。`law` で何を主張し、証明がどの仮定を使ったのかを確認します。断定できる範囲だけを読者へ伝えることが重要です。

演習　「盤面の地雷数が一定」と「ファイルへの保存が必ず成功する」が異なる種類の性質である理由を説明してみましょう。

Part V

並列実行とその他の応用

Chapter 19

CPU、GPU、Array.atomic

この章で学ぶこと：計算並列と入出力の並行を区別し、データの共有条件を理解する。

Bend の並列呼び出しでは、二つの独立した処理を同時に始める記法を使います。負荷が片方に偏ると待ち時間が増えます。

サンプル：均等な二分木での並列呼び出し

```
import Base

# Independent branches form a balanced fork/join computation.
def count_leaves(+level: Nat) -> U32:
  match level:
    case 0n:
      1
    case 1n+k:
      x y = count_leaves(k) count_leaves(k)
      (x + y : U32)

def main() -> U32:
  count_leaves(8n)
```

一方、関数呼び出しに! を付ける方法は GPU 側の実行指定です。環境によっては CPU で実行されます。必ず高速化するという意味ではありません。

`Array.atomic.add`、`Array.atomic.cas` などは共有されるデータの原子的更新に関わる API です。戻り値、対応する要素型、対象実行環境を公式 `Base` で確認してから使います。高速化の必要がないコードに無理に原子的操作を追加しません。

演習　独立した複数の盤面を並列に解析する場合、どの入力を分離する必要があるか考えてみましょう。

Chapter 20

C・JavaScript と外部連携

この章で学ぶこと：生成するプログラムとホスト環境の境界を知る。

Bend 2 は C のネイティブ実行ターゲットと JavaScript の実行ターゲットを持ちます。GPU やスレッドの振る舞い、使用できる外部機能はターゲット間で同じとは限りません。外部の C・JavaScript 実装を読み込む仕組みを使えば、Bend だけでは用意されていない操作を追加できます。

ただし、外部関数が返す結果や副作用は Bend 内の純粋な証明だけで保証できません。`@unsafe` などの安全性に関する境界とともに理解します。モジュールやパッケージの整理、ライセンス、BendHub 等を使った配布は、まず公式の最新ガイドで対応状況を確かめて進めます。

`Audio.open`、`Audio.write`、`Audio.close` も独立例として API の型を確認し、実際の音声出力は環境対応を確かめた場合だけ成功と記録します。

演習　同じサンプルを C ターゲットと JavaScript ターゲットで動かす際、何を別々に検証する必要があるか列挙してみましょう。

Part VI

マインスイーパー統合と発展

Chapter 21

一人で遊べるアプリにする

この章で学ぶこと：仕様、データ、画面、保存、テストを順番に統合する。

完成する通常版は、地雷配置、左クリックでの公開、右クリックでの旗の切替、0 マスの連鎖公開、勝敗判定、経過時間、再スタートを備えます。一手目が安全である条件も明示します。

1. 地雷配置と公開情報を別の型で保持する。
2. 公開、旗、勝敗判定を純粋関数で実装し、局所法則を書く。
3. `Image` へ盤面を描画し、`Event` から操作へ変換する。
4. 設定・戦績を `File` へ保存し、復元時は妥当性を検査する。
5. CLI の独立した期待値テストで規則を検証し、実機 GUI テストを追加する。

実際に統合する際は、一つずつ機能を追加し、テストで正しさを確認してから次の機能へ進めましょう。

演習　最初のクリック前の盤面と、地雷を配置した後の盤面を別の型として扱う利点を考えてみましょう。

Chapter 22

発展：運ゲー無しマインスイーパー

この章で学ぶこと：「公開情報から確実に分かること」を形式化し、ソルバーによる盤面検証と生成を理解する。

22.1　なぜ通常の盤面では推測が必要になるのか

通常のマインスイーパーでは、開いたマスに書かれた数字が同じでも、隠れている地雷の置き方が複数あり得ます。例えば、数字の 1 の周囲に未確定のマスが二つ残り、そのどちらにも地雷が置ける場合、片方を適当に開くことは推論ではありません。

運ゲー無し版では、**最初に開くマスが決まったあと、確実に安全と判定できる手だけを続けて勝利できる盤面**を採用します。Simon Tatham の『Mines』は、この考え方を使った公開実装です。実際にソルバーを用意し、候補の地雷配置を解かせて検証する方法が知られています。

22.2　最小の推論規則を Bend 2 で書く

開かれたマスに表示された数字を c、周囲で地雷と**確定した**マス数を f、まだ確定していないマス数を u とします。

$$r = c - f$$

は、未確定マスに残っている地雷数です。$0 \le r \le u$ が満たされるとき、次の二つの規則が使えます。

- $r = 0$：周囲の未確定マスはすべて安全です。
- $r = u$：周囲の未確定マスはすべて地雷です。

$0 < r < u$ であれば、その数字だけでは決まりません。$r > u$ は矛盾です。プレイヤーが旗を置いたマスを無条件で「確定した地雷」にしてはいけません。誤った旗は誤推論を引き起こすためです。

サンプル：局所制約の判定と法則

```
import Base

# Deduce from ONE visible number:
# remaining = displayed clue - mines already established as certain
# unknown   = number of still-unclassified adjacent squares
#
# "Flagged by the player" is NOT automatically "certain mine".
```

```
type LocalRule is Data:
  AllSafe{}
  AllMines{}
  Unknown{}
  Inconsistent{}

# Definition order matters: each helper is available to those below it.
# Computed conditions are passed as function parameters to pattern matches.
def infer_equal(remaining: U32, unknown: U32, equal: Bool) -> LocalRule:
  match equal:
    case True{}:
      AllMines{}
    case False{}:
      Unknown{}

def infer_zero(+remaining: U32, +unknown: U32, empty: Bool) -> LocalRule:
  match empty:
    case True{}:
      AllSafe{}
    case False{}:
      infer_equal(remaining, unknown, U32.is_eq(remaining, unknown))

def infer_limit(+remaining: U32, +unknown: U32, over: Bool) -> LocalRule:
  match over:
    case True{}:
      Inconsistent{}
    case False{}:
      infer_zero(remaining, unknown, U32.is_eq(remaining, 0))

def infer(+remaining: U32, +unknown: U32) -> LocalRule:
  infer_limit(remaining, unknown, U32.is_lt(unknown, remaining))

# A small observable numeric output for automated testing.
def tag(r: LocalRule) -> U32:
  match r:
    case AllSafe{}:
      1
    case AllMines{}:
      2
    case Unknown{}:
      3
    case Inconsistent{}:
      4

law zero_remaining_is_safe_for_three:
  {infer(0, 3) == AllSafe{} : LocalRule}

def zero_remaining_is_safe_for_three():
  {==}

def main() -> U32:
  a = tag(infer(0, 3))
  b = tag(infer(3, 3))
  c = tag(infer(1, 3))
  d = tag(infer(4, 2))
  (a + b + c + d : U32)
```

`infer` は残り地雷数と未確定マス数から結果を表す ADT を返します。`AllSafe` や `AllMines` は推論の種類を表し、マスの座標自体を返すものではありません。`Unknown` の場合に勝手に安全マスを決めてはいけません。入力に矛盾があるときは `Inconsistent` で区別します。付属の `law` は、残り地雷数が 0 で未確定マスが 3 個ある場合の具体例です。任意のマス数や盤面全体

を証明したものではありません。

演習 未確定マスが 4 個、残り地雷数が 0、2、4、5 の場合に、それぞれ `infer` が何を返すべきか調べてみましょう。

22.3 複数の数字から推論する

一つの数字だけで決まらない場合でも、複数の数字が制約する未確定マスの集合を比較すると、情報が得られることがあります。集合 A に地雷が a 個、集合 B に地雷が b 個という制約があり、$A \subseteq B$ なら、

$$\#\big((B \setminus A) \cap M\big) = b - a$$

です。特に $b = a$ なら $B \setminus A$ のマスはすべて安全です。$b - a = |B \setminus A|$ ならすべて地雷です。この**部分集合の規則**は、`Set` の活用例でもあります。

さらに、残り地雷の総数を使った推論もあります。独立した場所に分かれる制約でも、総地雷数によって片方の候補が排除されることがあります。標準ライブラリの集合・リスト・辞書をどう使うか検討するのが中級編としての目的です。

22.4 公開情報と真の地雷配置を分離する

公開状態 V から矛盾なく置ける地雷配置の集合を $\mathcal{C}(V)$ とします。ここには**開いたマスとその数字、盤面サイズ、総地雷数**だけが反映されます。未公開の本当の地雷位置は渡しません。

$\mathcal{C}(V) \neq \varnothing$ であり、未公開マス p がすべての候補で地雷でないなら、

$$\forall M \in \mathcal{C}(V), \quad p \notin M,$$

p は確実に安全です。候補集合が空の場合は、入力が矛盾しているものとしてエラーにします。空集合に対する全称命題を利用して「安全」と判定するのは誤りです。

この区別をコードで守ることが重要です。

- **ソルバー**：公開状態 V だけを受け取り、確実な安全マスを返す。
- **ゲーム本体**：真の配置を保持し、実際に開いたマスの数字を計算する。
- **検証器**：ソルバーの提案した操作をゲーム本体で再生し、最後まで安全にクリアできるか検査する。

22.5 候補生成と検証を繰り返す

初手 p_0 を決め、そのマスを必ず安全にするよう候補の地雷配置を作ります。より大きく開ける開始局面にしたい場合は、初手とその周囲 8 マスを地雷禁止領域にできます。その後、ソルバーに**公開状態だけ**を渡して確実な操作を繰り返させます。

1. 初手が安全な地雷配置を生成します。
2. 初手を開き、数字が 0 なら周囲を連鎖的に開きます。
3. 公開数字と総地雷数から、確実な安全マスを探します。
4. 安全マスが見つかれば開き、全安全マスを開けるまで続けます。

5. 候補が残っているのに確実な手がなければ、その盤面は採用しません。

生成の試行回数には上限を設けます。上限に達した場合は**生成失敗を明示**し、推測が必要な盤面を無断で運ゲー無し版として公開してはいけません。あらかじめ検証した盤面を別途用意して代替する方法もあります。

22.6 小盤面用の参考実装と検証

この節の発展用参考実装はリポジトリの 中級編の参考実装にあります。学習用 ZIP には完成コードを入れず、読者が必要に応じて参照できます。

小盤面用の Python 検証器は、公開数字と総地雷数に矛盾しない候補配置を**全列挙**し、そのすべてで安全なマスだけを開きます。列挙数が上限を超えた場合は採用を保留します。これにより、限定した盤面サイズでは、局所規則で分からない確定手まで探せます。

ただし、候補の全列挙は盤面が大きくなると指数的に重くなります。これは中級編の学習・テスト用の独立した参考実装であり、実用サイズの Bend 版ソルバーや GUI アプリの完成を意味しません。Bend への移植を進める際は、まず局所推論と集合の差分から取り組みましょう。

22.7 「運ゲー無し」の保証範囲

解が一意であることと、途中の局面で確実な手を選べることは異なります。また、局所規則しか知らないソルバーが手詰まりになっても、もっと強い推論で解ける可能性があります。したがって、ソルバーが最後まで解けた盤面だけを採用する方法は確実ですが、採用されなかった盤面が必ず運ゲーであるとは断言できません。

この教材では「**固定した初手から、このソルバーが公開情報のみで全安全マスを開ける**」ことを明示的な合格条件にします。初手が変わっても常に同じ条件を満たす、すべての推論規則を網羅している、すべての盤面を等確率で生成する、といった強い条件は主張しません。

参考資料

- Simon Tatham (2019), Writing a soluble-grid generator for Mines：候補の改変とソルバー検査。
- JSMinesweeper：推測不要盤面の生成、ソルバー、分析。
- Mindsweeper：公開されている別の無推測盤面生成の実装。

演習　公開状態と真の地雷配置を別の型で定義してみましょう。また、局所推論しか使わないソルバーが行き詰まるが、集合の差分を使えば先に進める例を考えてみましょう。

Chapter 23

API 索引と次の学習

この章で学ぶこと：使い方が分からない機能を自分で調べ、検証できるようにする。

標準ライブラリは機能名を列挙するだけでは使いこなせません。必要な操作について**引数・戻り値・所有権・失敗・対象ターゲット**を調べる方法を繰り返しましょう。

分類	主な名前	確認すること
数値	Nat, U32, F32, Word	変換と丸め、上限
文字	Char, String	分割、解析、文字境界
列・配列	List, Array	走査、複製、所有権
辞書・集合	Map, Set	文字列キー、検索結果
結果型	Maybe, Result, Either	失敗の情報
入出力	IO, File, Process	ハンドル、終了コード
通信	TCP, UDP, Socket	接続と切断
並行処理	Chan, IO.fork, IO.join	同期とタイムアウト
GUI	Image, App, Window, Event	画面・イベントの環境
音声	Audio	デバイスと書込み
証明	law, Equal, Sigma	仮定と結論、カーネル

公式資料：Bend 2 公式ガイド、公式 Base ライブラリ。

演習　初めて見る `Base` の関数を一つ選び、最小サンプルを作って型検査し、成功時と失敗時の振る舞いを確かめましょう。